

Breaking change in `std::span`'s new `initializer_list` constructor

Document #: P4144R0
Date: 2026-03-25
Project: Programming Language C++ LEWG
Reply-to: Mark Hoemmen
<mhoemmen@nvidia.com>

Contents

- 1 Author
- 2 Revision history
- 3 Problem: “Silent” change in `span`'s behavior
 - 3.1 Summary
 - 3.2 Cause
- 4 It's not “silent,” but implementations diverge
- 5 P2447 introduced wording bugs, later fixed
- 6 Proposed fix
 - 6.1 Ansatz: Constrain the `initializer_list` constructor
 - 6.2 Fix `initializer_list` construction from actual `bool` values
 - 6.3 Fix `initializer_list` construction from any non-pointer type convertible to `bool`?
- 7 Alternatives
- 8 Wording change
 - 8.1 Increment `__cpp_lib_span` feature test macro
 - 8.2 Change `[span.cons]`

§ 1 Author

- Mark Hoemmen (mhoemmen@nvidia.com) (NVIDIA)

§ 2 Revision history

§ 3 Problem: “Silent” change in span’s behavior

§ 3.1 Summary

This paper expresses [LWG4520](#) and proposes a fix.

LWG 4520 came from [an issue](#) filed on NVIDIA’s Standard Library’s (libc++) implementation of span.

We’ve written a demo (also with partial implementation of proposed fix) [here](#). Here is a short illustration of the change in behavior between C++23 and C++26.

```
#include <span>

int main() {
    bool data[4] = {true, false, true, false};
    bool* ptr = data;
    size_t size = 4;

    std::span<bool> span_nc{ptr, size};
    // OK in both C++23 and C++26
    assert(span_nc.size() == size);

    std::span<const bool> bad_span{ptr, size};
    // OK in C++23, but FAILS in C++26
    assert(bad_span.size() == size);
    return 0;
}
```

§ 3.2 Cause

In C++23, constructor `span(element_type*, size_t)` is called.

Adoption of [P2447R6](#) for C++26 means that constructor `span(initializer_list<value_type>)` is selected instead. It takes the conversion sequence from `bool*` to `bool` and `size_t` to `bool`.

P2447 authors understood that adoption of their proposal would be a breaking change. The paper adds to Annex C some examples of code that this breaks. This includes “silent” semantic changes to code, such as the following.

```
void *a[10];
// x is 2; previously 0
int x = span<void* const>{a, 0}.size();
any b[10];
// y is 2; previously 10
int y = span<const any>{b, b+10}.size();
```

The use case in [LWG4520](#) is most like the `span<const any>` break above, in that both `ElementType*` and `size_t` are convertible to the `span`'s `value_type`.

However, `span<const any>` is a less common use case. Creating a `span<const bool>` is likely more common, especially in generic code. That means LEWG might not have realized the significance of the change.

Note that `span<bool>` does not have this issue. Using `{}` with `mdspan` works just fine. (Historically, `mdspan` came first.)

§ 4 It's not “silent,” but implementations diverge

This is actually ill-formed code; it's a narrowing conversion from pointer to `bool`. GCC trunk compiles this but emits narrowing warnings. Clang stops with an error.

Per [\[intro.compliance.general\] 2.3](#), both implementations are conforming, as they emit a “diagnostic.” GCC implements a conforming “extension” per [\[intro.compliance.general\] 11](#), in that it attempts to give meaning (by compiling) to invalid code.

§ 5 P2447 introduced wording bugs, later fixed

Adoption of P2447 led to bugs in the Standard wording that later had to be fixed. GCC [Bug 120997](#) led to filing of [LWG 4293](#). The Standard was specified to use curly braces for the return values of some `span` member functions, such as `submdspan`. Adoption of [LWG 4293](#)'s Proposed Fix fixed that.

§ 6 Proposed fix

§ 6.1 Ansatz: Constrain the initializer list constructor

Constrain `span(initializer_list<value_type>)` constructor so that `value_type` is not `bool`.

```
constexpr
explicit(extent != dynamic_extent)
span(initializer_list<value_type>)
requires(
    is_const_v<ElementType &&
    (! is_same_v<value_type, bool>) // ADD
);
```

That would have the advantage that everything else works fine still, but we can avoid the breakage of valid user code, so only `span<const bool>` is affected.

```
// Works in current C++26 draft;
// fails to compile with the above change
std::span<const bool> span_from_bool{true, false, true};

// Work-around: Enclose initializer_list in ()
std::span<const bool> span_from_bool2({true, false, true});
assert(span_from_bool2.size() == 3u);

// Work-around: Enclose initializer_list in {}
std::span<const bool> span_from_bool3{{true, false, true}};
assert(span_from_bool3.size() == 3u);
```

§ 6.2 Fix initializer list construction from actual bool values

The above change would break existing code, specifically by making the following fail to compile.

```
std::span<const bool> span_from_bool{true, false, true};
```

We can fix this by adding a new `initializer_list` constructor overload specifically for `bool` input values.

```
// NEW CONSTRUCTOR
template<typename InitListType>
constexpr
explicit(extent != dynamic_extent)
my_span(initializer_list<InitListType> il)
requires (is_const_v<ElementType>
    && (is_same_v<value_type, bool>)
    && (is_same_v<InitListType, bool>)
);
```

Here is an [implementation](#) (thanks to Giuseppe D'Angelo! who suggests that this constructor could be a C++29 feature, since it's an extension, as code would move from breaking to non-breaking).

§ 6.3 Fix initializer list construction from any non-pointer type convertible to bool?

It would be excellent to permit non-pointer types that are convertible to `bool`. For example, this would permit the common case of initializing `bool` values from 0 and 1 `int` literals.

```
std::span<const bool> span_from_bool{1, 0, 1};
```

However, I'm not sure how to specify this. Merely replacing the `is_same_v<InitListType, bool>` constraint with `is_integral_v<InitListType>` does *not* work. Doing that results in `span<const bool>{0, 1, 0}` selecting the `initializer_list<value_type>` constructor, which fails with a hard error, because it

attempts to initialize the `const bool*` pointer with a `const int*` from the input `initializer_list<int>::const_iterator`.

§ 7 Alternatives

1. File a GCC bug to make GCC emit an error here.
2. Try to constrain `span<T>`'s constructor generically (for all `T`) so that it refuses to convert a pointer to `value_type`.

Regarding Option (1), Mattermost discussion during the meeting suggests that GCC is unlikely to change its current behavior.

Option (2) is higher risk because `span` has many constructors.

§ 8 Wording change

Text in blockquotes is not proposed wording, but rather instructions for generating proposed wording.

§ 8.1 Increment `__cpp_lib_span` feature test macro

In `[version.syn]`, increase the value of the `__cpp_lib_span` macro by replacing `YYMMML` below with the integer literal encoding the appropriate year (YYYY) and month (MM).

```
#define __cpp_lib_mdspan YYMMML // also in <mdspan>
```

§ 8.2 Change `[span.cons]`

Change the `initializer_list<value_type>` constructor of `span` in `[span.cons]` 21 as follows.

```
constexpr explicit(extent != dynamic_extent)
span(std::initializer_list<value_type> il);
```

21 *Constraints:* `is_const_v<element_type> && (! is same v<value type, bool>)` ~~is~~are true.

22 *Hardened preconditions:* If `extent` is not equal to `dynamic_extent`, then `il.size() == extent` is true.

23 *Effects:* Initializes `data_` with `il.data()` and `size_` with `il.size()`.

```
template<typename InitListType>
constexpr explicit(extent != dynamic_extent)
span(std::initializer_list<InitListType> il);
```

24 *Constraints:*

- (24.1) — `is_const_v<element_type>` is true,
- (24.2) — `is_same_v<value_type, bool>` is true, and
- (24.3) — `is_same_v<InitListType, bool>` is true.

25 *Hardened preconditions:* If `extent` is not equal to `dynamic_extent`, then `il.size() == extent` is true.

26 *Effects:* Initializes `data_` with `il.data()` and `size_` with `il.size()`.

```
constexpr span(const span& other) noexcept = default;
```